TASK – ANSIBLE AUTOMATION FOR WORDPRESS HOSTING WITH NGINX, PHP, MARIADB, SFTP, PHPMYADMIN AND SSL

Description

The WordPress hosting environment was automated on an AWS EC2 RHEL server using Ansible. The project used Ansible Roles to divide the configuration into separate components and Ansible Vault to protect sensitive information. Ansible Navigator was used to execute the complete automation.

Objective

1. Install Ansible and Ansible Navigator on the RHEL server.
2. Create an organized Ansible project.
3. Configure the inventory and main playbook.
4. Create separate reusable roles for the required services.
5. Automate the installation of Nginx, PHP, PHP-FPM and MariaDB.
6. Configure the website user and website directory.
7. Configure SELinux and Nginx.
8. Configure SFTP with a chroot environment.
9. Install and configure phpMyAdmin.
10. Install and configure WordPress.
11. Protect passwords and secrets using Ansible Vault.
12. Configure HTTPS using Certbot.
13. Execute the complete deployment using Ansible Navigator.
14. Verify the final server configuration.

Requirements

- AWS EC2 instance with RHEL
- Python 3
- pip
- Ansible
- Ansible Navigator
- Ansible Roles
- Ansible Vault
- Nginx
- PHP
- PHP-FPM
- MariaDB
- WordPress
- phpMyAdmin
- SFTP
- SELinux
- DuckDNS domain
- Let's Encrypt SSL certificate

Procedure

1. Check Python and Pip

Python and pip were checked before installing the Ansible tools.

python3 --version
pip3 --version

These tools were required for installing Ansible Navigator and supporting the automation environment.

2. Install Ansible

The required packages for Ansible automation were installed on the RHEL server.

sudo dnf install -y python3 python3-pip podman ansible-core

The Ansible installation was verified using:

ansible –version

```
[ec2-user@ip-172-31-15-254 ~]$ sudo dnf install -y podman python3-pip
Updating Subscription Management repositories.
Unable to read consumer identity

This system is not registered with an entitlement server. You can use "rhc" or "subscription-manager" to register.

Last metadata expiration check: 0:01:53 ago on Tue Oct  6 04:33:45 2026.
Package podman-7:5.8.2-1.el10_2.x86_64 is already installed.
Dependencies resolved.
================================================================================
 Package            Architecture   Version                 Repository                          Size
================================================================================
Installing:
 python3-pip        noarch         23.3.2-11.el10_2        rhel-10-appstream-rhui-rpms        3.2 M
Upgrading:
 podman             x86_64         7:5.8.2-9.el10_2        rhel-10-appstream-rhui-rpms         15 M
```

```
REDHAT_SUPPORT_PRODUCT_VERSION="10.2"
[ec2-user@ip-172-31-15-254 ~]$ sudo dnf install ansible-core -y
Updating Subscription Management repositories.
Unable to read consumer identity

This system is not registered with an entitlement server. You can use "rhc" or "subscription-manager" to register.

Red Hat Enterprise Linux 10 for x86_64 - AppStream from RHUI (RPMs)     52 MB/s | 6.7 MB     00:00
Red Hat Enterprise Linux 10 for x86_64 - BaseOS from RHUI (RPMs)       133 MB/s |  97 MB     00:00
Red Hat Enterprise Linux 10 Client Configuration                        24 kB/s | 1.3 kB     00:00
Dependencies resolved.
================================================================================
 Package                Arch       Version                    Repository                  Size
================================================================================
Installing:
 ansible-core           noarch     1:2.16.16-2.el10_2.1       rhel-10-appstream-rhui-rpms 2.9 M
Installing dependencies:
 git-core               x86_64     2.52.0-1.el10              rhel-10-appstream-rhui-rpms 5.2 M
 python3-cffi           x86_64     1.16.0-7.el10              rhel-10-baseos-rhui-rpms    312 k
 python3-cryptography   x86_64     43.0.0-4.el10_2.5          rhel-10-baseos-rhui-rpms    1.4 M
```

3. Install Ansible Navigator

Ansible Navigator was installed using Python pip.

python3 -m pip install --user ansible-navigator

The local Python binary directory was added to the PATH:

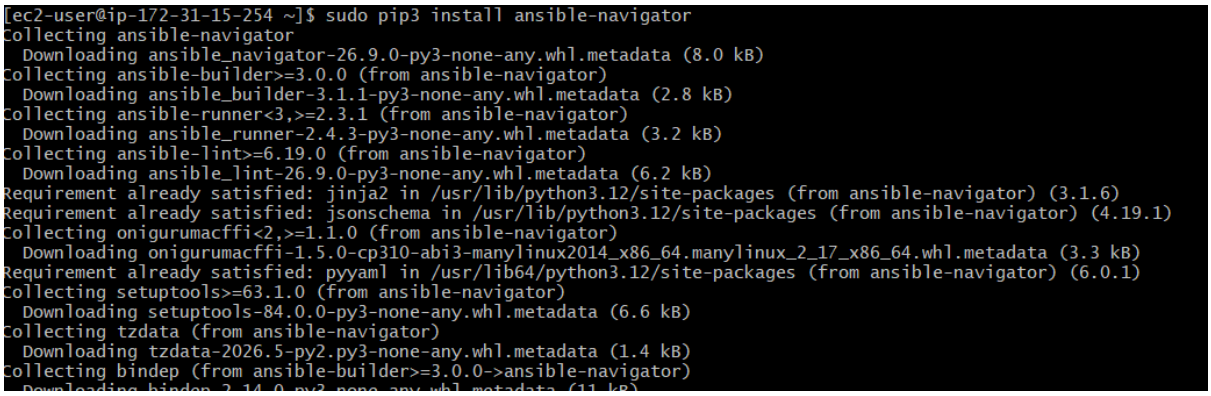
export PATH="$HOME/.local/bin:$PATH"

The PATH was made permanent using:

echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

The Navigator installation was verified using:

ansible-navigator --version



4. Create the Ansible Project

A separate directory was created for the WordPress automation project.

mkdir -p ~/wordpress-ansible
cd ~/wordpress-ansible

The project contains the main playbook, inventory, configuration file and roles.

The project structure was organized as:

wordpress-ansible/
├── site.yml
├── inventory
├── ansible.cfg
├── ansible-navigator.log
└── roles/
    ├── packages/
    ├── website/
    ├── sftp/
    ├── phpmyadmin/
    ├── wordpress/
    └── ssl/

5. Configure the Ansible Inventory

The inventory file was used to define the target server.

The EC2 server was configured as the local target:

[webservers]
localhost ansible_connection=local

Since Ansible was installed on the same EC2 server, the playbook was executed directly on the local system.

```
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ vi inventory
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ cat inventory
[webservers]
localhost ansible_connection=local
```

6. Configure Ansible

The project uses an ansible.cfg file for the Ansible configuration.

The active configuration file was located at:

/home/ec2-user/wordpress-ansible/ansible.cfg

This file provides the project-level Ansible settings and inventory configuration.

```
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ cat ansible.cfg
[defaults]
inventory = ./inventory
roles_path = ./roles
[ec2-user@ip-172-31-15-254 wordpress-ansible]$
```

7. Create the Main Playbook

The main playbook was created as:

site.yml

It acts as the entry point for the complete automation.

The roles were included in the following order:

packages
website
sftp
phpmyadmin
wordpress
ssl

This allows the server to be prepared first and the application and security configuration to be completed afterward.

```
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ cat site.yml
---
-  name: Configure Wordpress Server
   hosts: webservers
   become: true


   roles:
     - packages
     - website
     - sftp
     - phpmyadmin
     - wordpress
     - ssl
```

Ansible Roles

Created roles:

```
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ansible-galaxy init roles/packages
- Role roles/packages was created successfully
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ls roles/packages
README.md  defaults  files  handlers  meta  tasks  templates  tests  vars
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ansible-galaxy init roles/website
- Role roles/website was created successfully
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ls roles/website
README.md  defaults  files  handlers  meta  tasks  templates  tests  vars
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ansible-galaxy init roles/sftp
- Role roles/sftp was created successfully
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ls roles/sftp
README.md  defaults  files  handlers  meta  tasks  templates  tests  vars
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ansible-galaxy init roles/phpmyadmin
- Role roles/phpmyadmin was created successfully
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ls roles/
packages  phpmyadmin  sftp  website
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ls roles/p
packages/   phpmyadmin/
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ls roles/phpmyadmin
README.md  defaults  files  handlers  meta  tasks  templates  tests  vars
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ansible-galaxy init roles/wordpress
- Role roles/wordpress was created successfully
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ls roles/w
website/   wordpress/
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ls roles/wordpress/
README.md  defaults  files  handlers  meta  tasks  templates  tests  vars
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ansible-galaxy init roles/ssl
- Role roles/ssl was created successfully
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ ls roles/ssl/
README.md  defaults  files  handlers  meta  tasks  templates  tests  vars
```

8. Packages Role

The packages role was created to install the software required for the WordPress hosting environment.

The role handles the installation of:

Nginx
PHP
PHP-FPM
MariaDB

The required services are also started and enabled using Ansible.

The purpose of this role is to prepare the RHEL server with all the basic software required for the website.

```
[ec2-user@ip-172-31-15-254 wordpress-ansible]$  cat -n roles/packages/tasks/main.yml
     1  #SPDX-License-Identifier: MIT-0`
     2  # tasks file for roles/packages
     3
     4
     5  ---
     6  - name: Install Nginx
     7    ansible.builtin.dnf:
     8      name: nginx
     9      state: present
    10
    11  - name: Install PHP and PHP-FPM
    12    ansible.builtin.dnf:
    13      name:
    14        - php
    15        - php-fpm
    16        - php-mysqlnd
    17        - php-cli
    18        - php-curl
    19        - php-gd
    20        - php-mbstring
    21        - php-xml
    22        - php-zip
    23      state: present
    24
    25
    26
    27  - name: Install MariaDB server
    28    ansible.builtin.dnf:
    29      name: mariadb-server
    30      state: present
    31
    32
    33  - name: Start and enable required services
    34    ansible.builtin.systemd:
    35      name: "{{ item }}"
    36      state: started
    37      enabled: true
    38    loop:
    39      - nginx
    40      - php-fpm
    41      - mariadb
```

9. Website Role

The website role was created to prepare the website environment.

A Linux user named:

zenhal

was created.

The website root directory was created at:

/home/zenhal/blog/public

The role also handles:

- Website directory creation
- Ownership and permissions
- SELinux configuration
- Nginx virtual host configuration
- Nginx configuration validation
- Nginx reload

The website directory was configured with the SELinux web-content context:

httpd_sys_content_t

This allows the web server to access the website while keeping SELinux enabled.

```
[ec2-user@ip-172-31-15-254 wordpress-ansible]$  cat -n roles/website/tasks/main.yml
     1  #SPDX-License-Identifier: MIT-0
     2
     3  # tasks file for roles/website
     4
     5  ---
     6  - name: Create webiste user
     7    ansible.builtin.user:
     8      name: zenhal
     9      home: /home/zenhal
    10      create_home: true
    11      state: present
    12
    13  - name: Create website root directory
    14    ansible.builtin.file:
    15      path: /home/zenhal/blog/public
    16      state: directory
    17      owner: zenhal
    18      group: zenhal
    19      mode: '0755'
    20
    21
    22  - name: Set SELinux context for website
    23    ansible.builtin.command:
    24      cmd: semanage fcontext -a -t httpd_sys_content_t "/home/zenhal/blog/public(/.*)?"
    25
    26  - name: Restore SELinux context
    27    ansible.builtin.command:
    28      cmd: restorecon -Rv /home/zenhal/blog/public
```

10. Nginx Configuration

As part of the Website role, Ansible created the Nginx virtual host configuration.

The domain used was:

zenhal.duckdns.org

The website root was configured as:

/home/zenhal/blog/public

PHP requests were configured to use PHP-FPM through:

/run/php-fpm/www.sock

WordPress routing was configured using the Nginx try_files directive.

The Nginx configuration was validated using:

nginx -t

After validation, Nginx was reloaded.

11. SFTP Role

The sftp role was created to provide SFTP access for the zenhal user.

SSH password authentication was enabled.

The SFTP configuration included:

PasswordAuthentication yes
ForceCommand internal-sftp
ChrootDirectory /home/zenhal
AllowTcpForwarding no
X11Forwarding no

The configuration restricts the zenhal user to the SFTP environment instead of providing normal shell access.

12. SFTP Chroot Configuration

The SFTP chroot directory was configured as:

/home/zenhal

The chroot directory was assigned to:

root:root

with permissions:

755

The website directories inside the chroot remained owned by:

zenhal:zenhal

This allows the user to manage website files while restricting access to other system directories.

The SFTP service was then restarted through Ansible.

```
[ec2-user@ip-172-31-15-254 wordpress-ansible]$  cat -n roles/sftp/tasks/main.yml
     1  #SPDX-License-Identifier: MIT-0
     2  ---
     3  # tasks file for roles/sftp
     4
     5
     6  - name: Enable SSH password authentication
     7    ansible.builtin.lineinfile:
     8      path: /etc/ssh/sshd_config.d/50-cloud-init.conf
     9      regexp: '^PasswordAuthentication'
    10      line: 'PasswordAuthentication yes'
    11      create: true
    12
```

13. Ansible Vault

Ansible Vault was used to protect sensitive information used by the automation.

Encrypted variable files were maintained for the following roles:

roles/website/vars/main.yml
roles/wordpress/vars/main.yml
roles/phpmyadmin/vars/main.yml

Sensitive information included:

SFTP password hash
WordPress database password
WordPress administrator password
WordPress administrator email
phpMyAdmin Blowfish secret

The values were therefore kept separate from the normal task files.

The encrypted variables were loaded using multiple Vault IDs during execution:

--vault-id phpmyadmin@prompt
--vault-id wordpress@prompt
--vault-id website@prompt

14. phpMyAdmin Role

The phpmyadmin role was created to automate the installation and configuration of phpMyAdmin.

The role performs the following operations:

Create phpMyAdmin directory
Download phpMyAdmin
Extract phpMyAdmin
Set ownership
Remove the archive
Create config.inc.php

Configure authentication
Configure database host
Disable passwordless login
Configure Blowfish secret

The phpMyAdmin installation directory is:

/home/zenhal/blog/public/phpmyadmin

The authentication type was configured as:

cookie

The database host was configured as:

localhost

Passwordless login was disabled.

The Blowfish secret was obtained from the encrypted Vault variable.

```
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ cat -n roles/phpmyadmin/tasks/main.yml
     1  #SPDX-License-Identifier: MIT-0
     2  ---
     3  # tasks file for roles/p
     4
     5
     6  - name: Create phpMyAdmin directory
     7    ansible.builtin.file:
     8      path: /home/zenhal/blog/public/phpmyadmin
     9      state: directory
    10      owner: zenhal
    11      group: zenhal
    12      mode: '0755'
    13
    14  - name: Download phpMyAdmin
    15    ansible.builtin.get_url:
    16      url: https://files.phpmyadmin.net/phpMyAdmin/5.2.3/phpMyAdmin-5.2.3-all-languages.tar.gz
    17      dest: /home/zenhal/blog/public/phpMyAdmin-5.2.3-all-languages.tar.gz
    18      mode: '0644'
    19
    20  - name: Extract phpMyAdmin
    21    ansible.builtin.unarchive:
```

15. WordPress Role

The wordpress role was created to automate the complete WordPress installation.

The role handles:

Create WordPress database
Create database user
Grant database privileges
Download WordPress
Extract WordPress
Set ownership
Create wp-config.php
Configure database details
Install WP-CLI

Check WordPress installation
Install WordPress
Configure site title
Configure tagline
Activate theme
Create personal blog post

The WordPress database configuration was:

Database Name: wordpress
Database User: zenhal
Database Host: localhost

The database password was loaded from the encrypted Vault file.

```
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ cat -n roles/wordpress/tasks/main.yml
     1  ---
     2  - name: Check WordPress installation
     3    ansible.builtin.command:
     4      cmd: /usr/local/bin/wp core is-installed --path=/home/zenhal/blog/public --allow-root
     5    register: wordpress_installed
     6    failed_when: false
     7    changed_when: false
     8
     9  - name: Install WordPress
    10    ansible.builtin.command:
    11      cmd: >
    12        /usr/local/bin/wp core install
    13        --path=/home/zenhal/blog/public
    14        --url=https://zenhal.duckdns.org
    15        --title="Zenhal Cloud & DevOps Blog"
```

16. Install WordPress

WordPress was downloaded and extracted automatically into:

/home/zenhal/blog/public

The WordPress files were assigned to:

zenhal:zenhal

The wp-config.php file was created from the sample configuration file.

The database settings were automatically configured as:

DB_NAME = wordpress
DB_USER = zenhal
DB_PASSWORD = Vault variable
DB_HOST = localhost

17. Install and Use WP-CLI

WP-CLI was installed by Ansible at:

/usr/local/bin/wp

The full path was used for WordPress management.

The WordPress installation was checked using:

```
/usr/local/bin/wp core is-installed \
  --path=/home/zenhal/blog/public \
  --allow-root
```

WordPress was installed through WP-CLI when required.

The site was configured with:

Site Title:
Zenhal Cloud & DevOps Blog

Tagline:
Cloud & DevOps Learning Journey

The Twenty Twenty-Five theme was activated.

A personal post titled:

My Cloud & DevOps Learning Journey

was also created.

18. SSL Role

The ssl role was created to automate HTTPS configuration.

The role:

Creates the Python environment
Installs Certbot
Installs the Certbot Nginx plugin
Requests the SSL certificate
Configures Nginx for HTTPS

The certificate was requested for:

zenhal.duckdns.org

The certificate files were stored under:

/etc/letsencrypt/live/zenhal.duckdns.org/

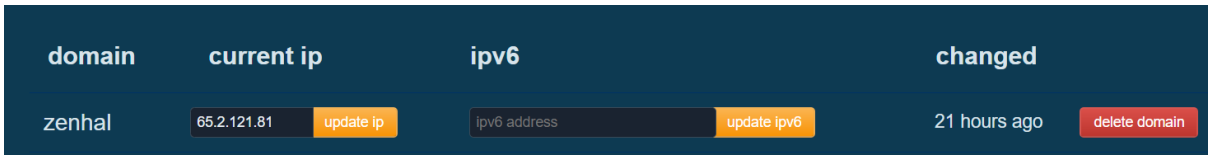
19. DuckDNS Configuration

The DuckDNS hostname used in the project was:

zenhal.duckdns.org

The hostname was configured to point to the EC2 public IP address.

This hostname was used for the Nginx virtual host and the SSL certificate.



Ansible Validation and Execution

20. Check the Main Playbook

Before executing the deployment, the complete playbook was checked for syntax errors.

The command used was:

```
ansible-playbook site.yml --syntax-check \
  --vault-id phpmyadmin@prompt \
  --vault-id wordpress@prompt \
  --vault-id website@prompt
```

The syntax check completed successfully and displayed:

playbook: site.yml

21. Test Ansible Navigator

A small test playbook named:

navigator-test.yml

was created to verify Navigator execution.

The test used the Ansible ping module against:

localhost

The result was:

ok: [localhost]

This confirmed that Ansible Navigator could execute tasks correctly on the EC2 server.

22. Execute the Complete Playbook Using Ansible Navigator

The complete automation was executed using:

```
ansible-navigator run site.yml --ee false -m stdout --enable-prompts \
  --vault-id phpmyadmin@prompt \
  --vault-id wordpress@prompt \
  --vault-id website@prompt
```

The command runs the site.yml playbook and executes all the roles.

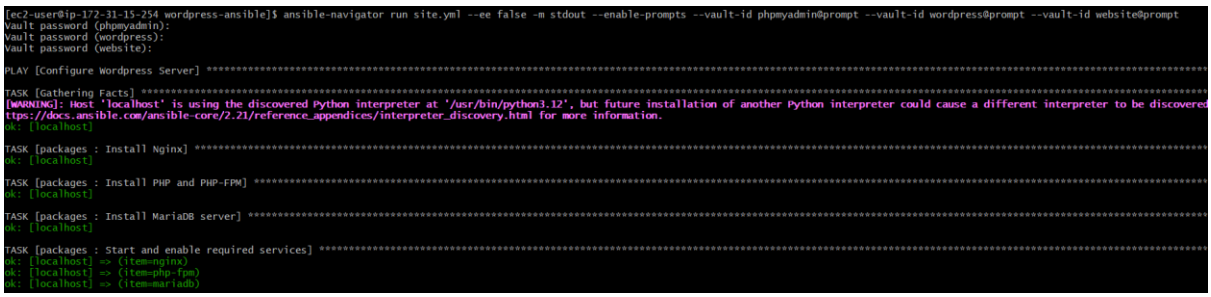
The execution flow was:

site.yml

```
├── packages
├── website
├── sftp
├── phpmyadmin
├── wordpress
└── ssl
```

The --ee false option was used so that Ansible executed directly on the RHEL EC2 server.

The Vault IDs were used to load the encrypted variables required by the roles.



23. Final Verification

After the playbook execution, the major services were verified.

Nginx:

sudo systemctl status nginx

PHP-FPM:

sudo systemctl status php-fpm

MariaDB:

sudo systemctl status mariadb

Listening ports:

sudo ss -tlnp

The web server was confirmed to be listening on:

Port 80
Port 443

The local web server response was checked using:

curl -I http://localhost

The server returned:

HTTP/1.1 200 OK

The WordPress database connection was checked using:

```
sudo /usr/local/bin/wp db check \
  --path=/home/zenhal/blog/public \
  --allow-root
```

The result was:

Success: Database checked.

SFTP was tested using:

sftp zenhal@SERVER_IP

The SFTP connection was successfully established.

```
drwxr-xr-x. 3 root root 74 Oct  6 06:54 /home/zenhal
[ec2-user@ip-172-31-15-254 wordpress-ansible]$ sftp zenhal@65.2.121.81
The authenticity of host '65.2.121.81 (65.2.121.81)' can't be established.
ED25519 key fingerprint is SHA256:4MbCEE57zCqDEQcq7Y2hObswwM3MWXr8sJ3L8JxVpxA.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:1: localhost
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '65.2.121.81' (ED25519) to the list of known hosts.
zenhal@65.2.121.81's password:
Connected to 65.2.121.81.
sftp>
```

24. Final Ansible Result

The complete Ansible deployment finished successfully with:

localhost : ok=54 changed=25 failed=0 skipped=1

The result confirmed that the complete automation executed successfully without failed tasks.

```
TASK [ssl : Install Python virtual environment package] ************************************************************
ok: [localhost]

TASK [ssl : Create Certbot virtual environment] ********************************************************************
changed: [localhost]

TASK [ssl : Install Certbot] ***************************************************************************************
changed: [localhost]

TASK [ssl : Install Certbot Nginx plugin] **************************************************************************
changed: [localhost]

TASK [ssl : Obtain SSL certificate and configure Nginx] ************************************************************
changed: [localhost]

PLAY RECAP *********************************************************************************************************
localhost                  : ok=54   changed=25   unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
```

URLs

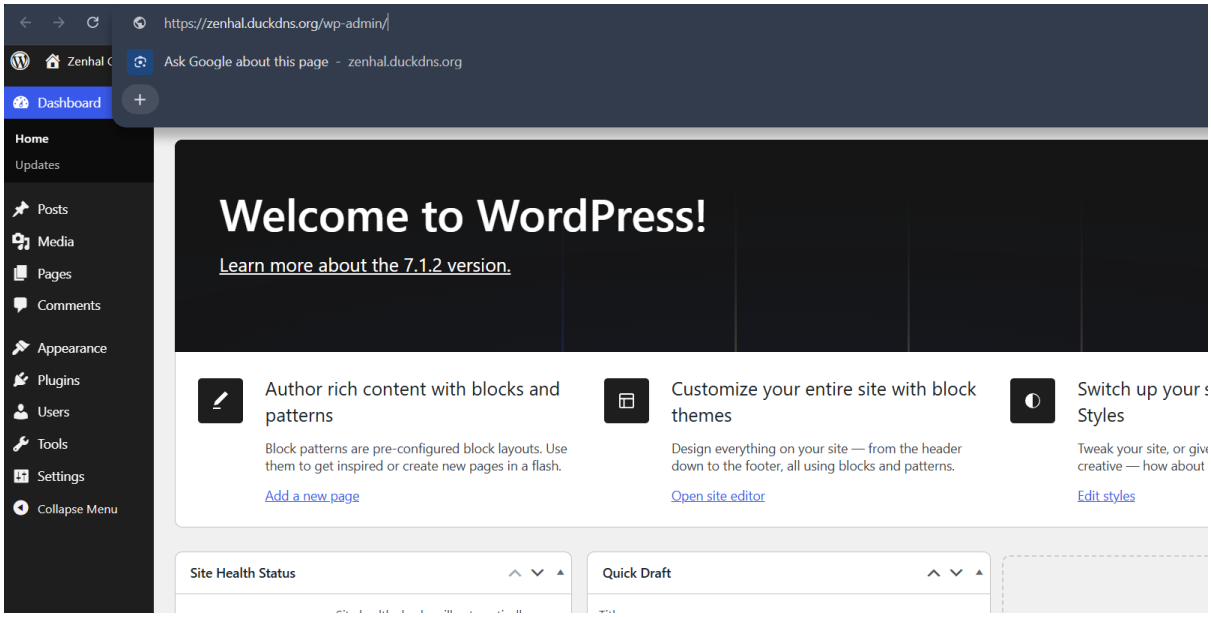
WordPress website:

https://zenhal.duckdns.org



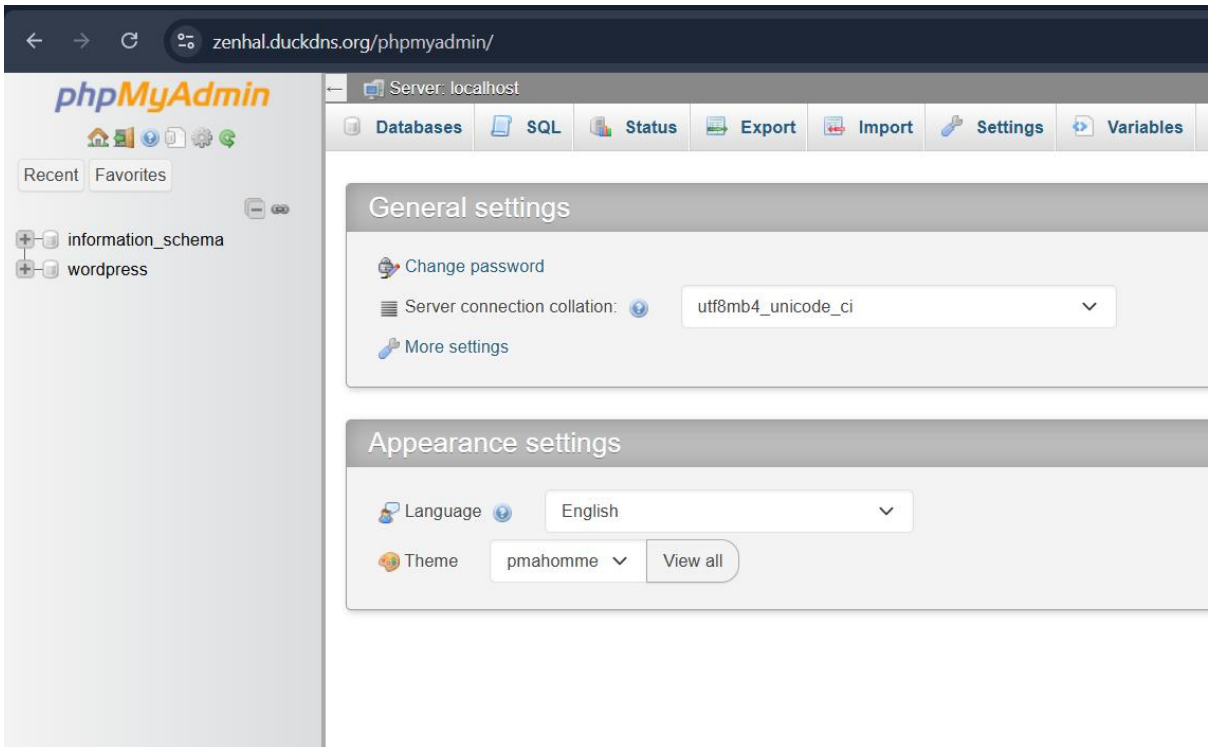
WordPress administrator:

https://zenhal.duckdns.org/wp-admin



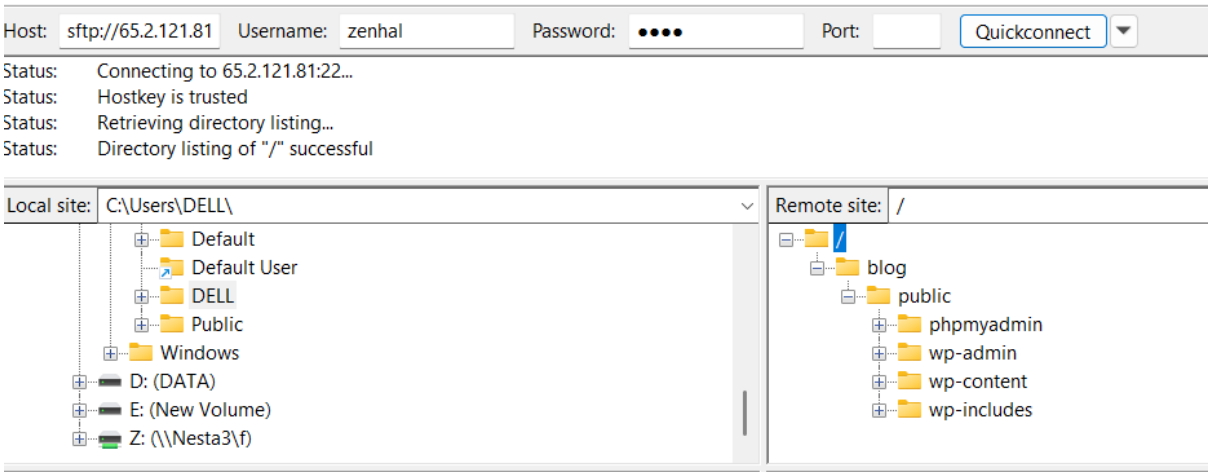
phpMyAdmin:

https://zenhal.duckdns.org/phpmyadmin



SFTP:

Host: [EC2 PUBLIC IP]
Port: 22
Username: zenhal
Protocol: SFTP



Credentials

SFTP Credentials

Host: [EC2 PUBLIC IP]
Username: zenhal
Password: [SFTP PASSWORD]
Port: 22
Protocol: SFTP

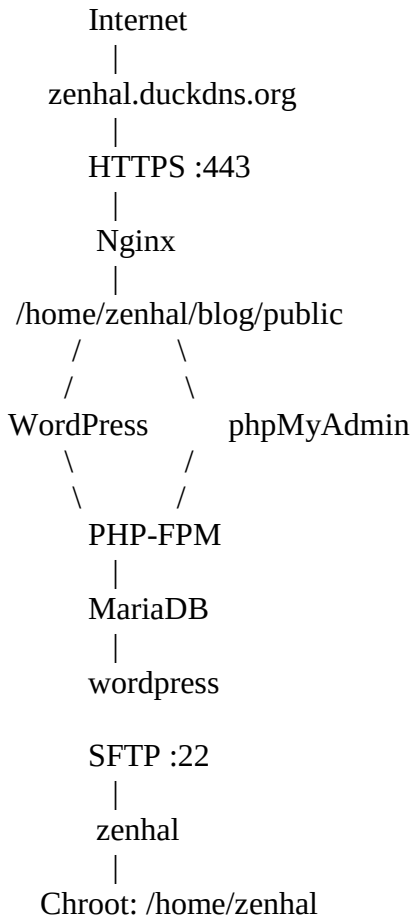
phpMyAdmin Credentials

URL: https://zenhal.duckdns.org/phpmyadmin
Username: zenhal
Password: [DATABASE PASSWORD]

WordPress Credentials

Admin URL: https://zenhal.duckdns.org/wp-admin
Username: zenhal
Password: [WORDPRESS ADMIN PASSWORD]

Final Architecture



Result

The WordPress hosting environment was successfully automated on an AWS RHEL EC2 server using Ansible.

The project was divided into separate roles for:

- Packages
- Website
- SFTP
- phpMyAdmin
- WordPress
- SSL

Ansible Vault was used to secure sensitive information, while Ansible Navigator was used to execute the complete playbook.

The final automation configured the web server, PHP environment, MariaDB database, WordPress, phpMyAdmin, SFTP, SELinux and SSL.

The complete execution finished with:

ok=54
changed=25
failed=0
skipped=1

Conclusion

This task demonstrated the use of Ansible for automating a complete WordPress hosting environment on an AWS EC2 RHEL server.

Using separate roles made the project modular and organized. Ansible Vault provided secure storage for passwords and secrets, while Ansible Navigator provided the interface for running the complete automation.

The project provided practical experience in Linux server automation, configuration management, web hosting, database management, SFTP, SELinux, SSL and DevOps practices.